Projet 1 – Automates
Clone de `egrep` avec support partiel des ERE

Saad Maïssa Dieme Ndeye Awa

DAAR – 2026–2027

1 Introduction

La commande UNIX `egrep` cherche un motif dans un fichier texte et affiche toutes les lignes qui le contiennent. Le motif est décrit par une *expression régulière*, c'est-à-dire une formule qui représente un ensemble de mots : par exemple, `chat|chien` représente les mots « chat » et « chien ».

Dans ce projet, nous avons écrit en Python un clone simplifié de `egrep`, qui accepte une partie des expressions régulières étendues (ERE). Le programme utilise un *automate* : une machine qui lit le texte caractère par caractère et indique s'il contient le motif. Nous suivons la méthode du cours (Aho et Ullman [1]) : l'expression est transformée en arbre, puis en automate non déterministe, puis en automate déterministe, qui est ensuite réduit au minimum. Quand le motif est un mot simple, sans opérateur, le programme utilise plutôt l'algorithme de Knuth-Morris-Pratt (KMP) [5], plus rapide dans ce cas. Le rapport suit le chemin d'un motif dans le programme, des opérateurs acceptés jusqu'aux mesures de vitesse face à `egrep`.

2 Présentation du problème

Le programme reçoit une expression régulière R et un fichier texte. Il affiche chaque ligne du fichier qui contient au moins une sous-chaîne appartenant au langage $L(R)$. Comme `egrep`, il renvoie le code de retour 0 si au moins une ligne correspond, 1 sinon, et 2 en cas d'erreur (expression mal formée, fichier introuvable ; la section 4 précise les écarts avec `egrep`).

Sous-ensemble supporté. Les lettres (tout caractère ASCII qui n'est pas un opérateur), le point "`.`" (n'importe quel caractère), la concaténation (implicite), l'alternative "`|`", les parenthèses, l'étoile "`*`" (zéro occurrence ou plus) et, en extension, le plus "`+`" (une occurrence ou plus), réécrit $X^+ = XX^*$. Priorité, du plus fort au moins fort : étoile et plus, concaténation, alternative. Ainsi `ab*|c` se lit $(a \cdot b^*) \mid c$.

Notations. La notation $O(\dots)$, appelée *complexité*, indique comment le temps de calcul augmente quand les données grandissent. Nous utilisons : m la longueur du motif, n le nombre d'états d'un automate, k la taille de l'alphabet et ℓ la longueur d'une ligne du fichier.

3 Architecture générale

La figure 1 montre le chemin suivi par un motif. Le programme regarde d'abord s'il contient des opérateurs : sinon, c'est un mot simple et la recherche utilise KMP ; sinon, le motif passe par toutes les étapes de construction de l'automate.

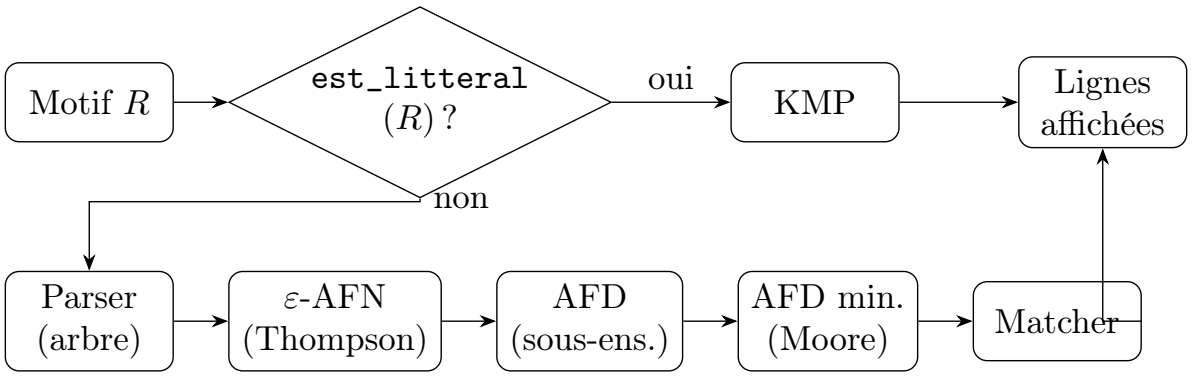


FIGURE 1 – Chemin suivi par un motif

Chaque étape est écrite dans un fichier Python séparé, ce qui permet de la tester seule (table 1). `myegrep.py` joue le rôle de `egrep` : il lit le fichier ligne par ligne, affiche les lignes trouvées et renvoie le code de retour. Il l'ouvre avec `newline=""` pour garder les fins de ligne d'origine (section 7).

Fichier	**Rôle**	**Complexité**
`parser.py`	expression → arbre	$O(m)$
`nfa.py`	arbre → ε-AFN (Thompson)	$O(m)$, au plus $2m$ états
`dfa.py`	ε-AFN → AFD (sous-ensembles)	jusqu'à 2^n états
`minimize.py`	réduction de l'AFD (Moore)	$O(n^2 k)$
`matcher.py`	recherche dans une ligne avec l'AFD	$O(\ell^2)$ au pire
`kmp.py`	recherche d'un mot simple (KMP)	$O(m + \ell)$
`myegrep.py`	lecture du fichier, choix de la méthode	–

TABLE 1 – Fichiers du programme

Structures de données. L'arbre est formé de nœuds (type, lettre, enfant gauche, enfant droit). L'ε-AFN est une liste de transitions (départ, symbole, arrivée), avec les symboles spéciaux `EPS` et `DOT`. L'AFD est un dictionnaire (état, symbole) → état, avec un ensemble d'états finaux : un pas de lecture coûte $O(1)$.

Choisir entre KMP et l'automate. Construire un automate prend du temps (l'arbre, puis trois automates successifs) et la recherche peut coûter $O(\ell^2)$: c'est beaucoup de travail inutile pour un mot simple. KMP prépare le motif en $O(m)$, puis lit chaque ligne une seule fois, sans jamais revenir en arrière : $O(\ell)$. La fonction `est_litteral` vérifie que le motif ne contient aucun des caractères `| * + . ( )` : `Sargon` utilise KMP, `S(a|o)rgon` utilise l'automate. Limite : `(ab)c` passe par l'automate alors qu'il décrit seulement le mot « abc » ; le résultat reste correct, mais plus lent (amélioration en section 9).

4 Analyse syntaxique

L'*analyse syntaxique* lit l'expression régulière et en déduit sa structure : un *arbre syntaxique*, où chaque nœud est un opérateur et chaque feuille une lettre. Il utilise cinq types de nœuds : `CHAR`, `DOT`, `CONCAT` et `ALT` (deux enfants), `STAR` (un enfant). La figure 2 montre l'arbre de `ab|cb`, exemple utilisé dans la suite.

Grammaire. Une *grammaire* est un ensemble de règles qui décrivent comment une expression est construite. La nôtre a quatre règles, une par niveau de priorité :

$$alt \to concat\ (\texttt{|}\ concat)^* \qquad concat \to etoile\ etoile^*$$
$$etoile \to atome\ (\texttt{*} \mid \texttt{+})^* \qquad atome \to lettre \mid \texttt{.} \mid \texttt{(}\ alt\ \texttt{)}$$

Plus une règle est basse, plus son opérateur est prioritaire. Les parenthèses renvoient à la première règle : c'est pourquoi elles permettent de changer la priorité.

Descente récursive. Les règles deviennent trois fonctions Python : `parse_alt` (alt), `parse_concat` (concat et etoile, car `*` et `+` sont traités juste après chaque atome) et `parse_atome` (atome). Chacune appelle celle du niveau suivant ; un curseur avance de gauche à droite sans jamais reculer, donc l'analyse coûte $O(m)$. Voici `parse_concat`, en pseudo-code. La concaténation n'a pas de symbole : le parser la reconnaît quand un élément est directement suivi d'un autre.

```
fonction parse_concat():
    noeud <- vide
    tant que le caractere courant n'est ni '|', ni ')', ni la fin :
        element <- parse_atome()
        tant que le caractere courant est '*' ou '+' :
            element <- STAR(element)      # '+' : CONCAT(element, STAR(element))
        noeud <- element si noeud est vide, sinon CONCAT(noeud, element)
    retourner noeud       # erreur si rien n'a ete lu
```

Le `+` est remplacé pendant la lecture : X^+ devient $X \cdot X^*$, ce qui évite un nouveau type de nœud. Inconvénient : le sous-arbre X est copié deux fois ; avec des `+` imbriqués, comme dans `((a+)+)+`, la taille de l'arbre double à chaque niveau. Ce cas est rare.

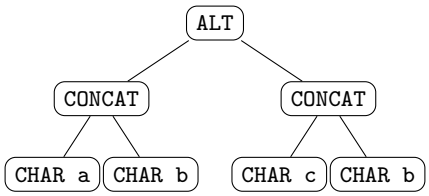


FIGURE 2 – Arbre syntaxique de `ab|cb`, exemple utilisé dans la suite

Erreurs. Notre analyseur est plus strict que `egrep`. Il renvoie le code 2 pour : `*a` (l'étoile n'a rien à répéter), `a|` et `()` (il manque une expression), `(ab` (parenthèse non fermée), `ab)` (parenthèse sans ouvrante) et l'expression vide. `egrep` ne signale une erreur que pour `(ab` : il accepte les autres, en lisant `a|` et `()` comme une alternative ou un groupe vides, qui reconnaissent le mot vide et donc toutes les lignes. Nous avons choisi de les rejeter, car ce sont presque toujours des fautes de frappe. En revanche, `a**` est accepté, comme dans `egrep`, car $(X^*)^* = X^*$.

5 Construction des automates

Un *automate* est un ensemble d'*états* reliés par des *transitions* (« dans cet état, en lisant cette lettre, aller dans cet autre état »). Il commence dans un *état initial* ; si, après avoir lu un mot, il est dans un *état final*, le mot est reconnu.

5.1 Construction de l'ε-AFN (Thompson)

Une *ε-transition* est un passage « gratuit » (sans lire de caractère) ; *non déterministe* veut dire que plusieurs chemins sont possibles en même temps. La construction de Thompson [2] (cours, Aho et Ullman [1]) parcourt l'arbre et construit un petit automate par nœud, avec un gabarit fixe :

— **lettre** `a` : deux états reliés par une transition `a` ;
— **concaténation** XY : une ε-transition relie l'état final de X à l'état initial de Y ;
— **alternative** $X|Y$: un nouvel état initial mène par ε vers X et vers Y, dont les états finaux mènent par ε vers un nouvel état final ;
— **étoile** X^* : un nouvel état initial et un nouvel état final ; une ε-transition permet de sauter X (zéro répétition), une autre de revenir au début de X (répéter).

Chaque nœud ajoute au plus deux états : l'automate a au plus $2m$ états et se construit en $O(m)$. La figure 3 montre le résultat pour `ab|cb` (10 états). Pour simuler cet automate, on suit tous les chemins en même temps, avec `epsilon_fermeture(E)` (ajoute à l'ensemble E les états accessibles par ε) et `transiter(E, c)` (états atteints depuis E en lisant c).

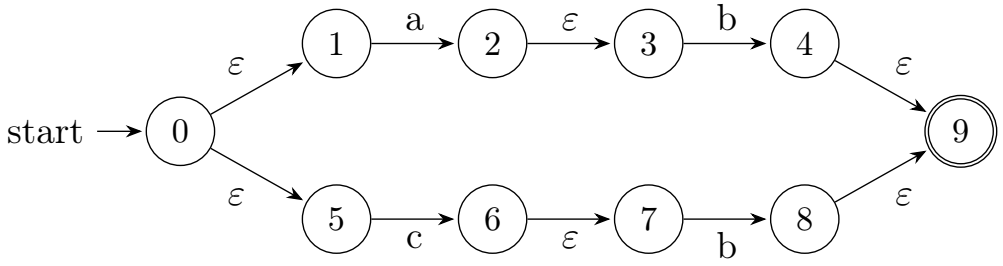


FIGURE 3 – ε-AFN de `ab|cb` (construction de Thompson)

5.2 Déterminisation (méthode des sous-ensembles)

Simuler un automate non déterministe est lent : à chaque caractère, il faut mettre à jour tout un ensemble d'états. On construit donc un *AFD* (automate fini déterministe) : depuis chaque état, et pour chaque caractère, il y a au plus un chemin. **Un état de l'AFD représente un ensemble d'états de l'ε-AFN** ; dans le code, c'est un `frozenset` Python, utilisable comme clé de dictionnaire.

```
initial <- epsilon_fermeture({etat initial de l'AFN})
a_traiter <- [initial]
tant que a_traiter n'est pas vide :
    E <- retirer un element de a_traiter
    pour chaque symbole c de l'alphabet :
        F <- epsilon_fermeture(transiter(E, c))
        si F n'est pas vide :
            ajouter la transition E --c--> F
            si F est nouveau : ajouter F a a_traiter
un etat E est final s'il contient un etat final de l'AFN
```

Pour `ab|cb`, on obtient 5 états : $A = \{0, 1, 5\}$ (initial), $B = \{2, 3\}$, $C = \{6, 7\}$, $D = \{4, 9\}$ et $E = \{8, 9\}$ (finaux), avec $A \xrightarrow{a} B$, $A \xrightarrow{c} C$, $B \xrightarrow{b} D$ et $C \xrightarrow{b} E$ (figure 4, gauche).

Le caractère universel. L'alphabet ne peut pas contenir tous les caractères possibles : il contient les lettres présentes dans l'expression, plus un symbole spécial `AUTRE` qui représente tous les autres caractères. Une transition « point » existe pour chaque symbole, y compris `AUTRE`.

Complexité. Un ensemble de n états a 2^n sous-ensembles : l'AFD peut avoir jusqu'à 2^n états. Ce pire cas existe vraiment : pour `(a|b)*a(a|b)(a|b)`, l'AFD doit retenir les trois derniers caractères lus, et le nombre d'états double à chaque `(a|b)` ajouté. En pratique, pour des motifs courants, l'AFD reste petit.

5.3 Minimisation (algorithme de Moore)

L'AFD contient souvent des états qui « font la même chose » : dans la figure 4 (gauche), B et C attendent tous les deux un `b`, puis mènent à un état final. La *minimisation* fusionne ces états équivalents. L'algorithme de Moore [3] procède par *raffinement de partition* : on sépare les états finaux des autres ; on calcule pour chaque état sa *signature* (les groupes atteints avec chaque symbole) ; deux états d'un même groupe de signatures différentes sont séparés ; on recommence jusqu'à ce que plus aucun groupe ne change.

Sur `ab|cb`, les groupes de départ sont $\{A, B, C\}$ et $\{D, E\}$. Au tour 1, A (qui a des transitions a et c) est séparé de B et C, qui ont la même signature (`b` vers un groupe final) ; D et E, sans transition,

restent ensemble. Au tour 2, plus rien ne change. L'automate minimal a 3 états au lieu de 5 (figure 4, droite).

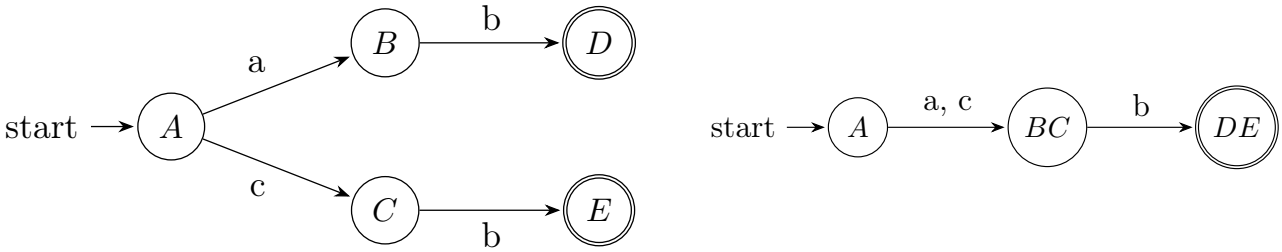


FIGURE 4 – AFD de `ab|cb` (5 états, à gauche) et AFD minimal (3 états, à droite)

Complexité. Chaque tour calcule n signatures de k symboles, avec au plus n tours : $O(n^2 k)$. L'algorithme de Hopcroft [4] fait le même travail en $O(k\, n \log n)$, mais il est plus difficile à écrire ; les AFD obtenus sont petits, nous avons choisi Moore.

6 Recherche dans les fichiers

Pour chaque ligne, le programme répond à une question : « cette ligne contient-elle un mot reconnu par le motif ? », n'importe où dans la ligne. La méthode (automate ou KMP) est choisie une seule fois, avant la lecture.

6.1 Recherche avec l'automate

L'AFD reconnaît un mot à partir de son *début*. Pour trouver un mot n'importe où dans la ligne, le matcher (`src/matcher.py`) essaie chaque position de départ, et s'arrête dès qu'il atteint un état final (il suffit de savoir que la ligne contient le motif) :

```
def cherche_depuis(dfa, ligne, i):
    etat = dfa.debut
    if etat in dfa.finaux:          # le motif accepte le mot vide
        return True
    j = i
    while j < len(ligne):
        etat = suivant(dfa, etat, ligne[j])
        if etat is None:            # pas de transition : echec
            return False
        if etat in dfa.finaux:      # motif trouve
            return True
        j = j + 1
    return False

def ligne_correspond(dfa, ligne):
    for i in range(len(ligne) + 1):
        if cherche_depuis(dfa, ligne, i):
            return True
    return False
```

`suivant` renvoie l'état atteint en lisant un caractère, ou `None` s'il n'y a pas de transition : aucun mot du motif ne commence alors à la position i.

Le mot vide. Un motif comme `a*` reconnaît le mot vide, qui est présent dans toutes les lignes, même vides : il doit donc afficher toutes les lignes, comme `egrep`. C'est pourquoi le matcher teste d'abord si l'état initial est final, et pourquoi la boucle va jusqu'à `len(ligne) + 1`.

Exemple. Avec `ab|cb` (automate minimal) et la ligne `xxcbz` : aux positions 0 et 1 (`x`), il n'y a pas de transition depuis A ; à la position 2, $A \xrightarrow{c} BC \xrightarrow{b} DE$, état final : la ligne est affichée.

Complexité. Il y a ℓ positions de départ, et chacune peut lire jusqu'à ℓ caractères : $O(\ell^2)$ par ligne dans le pire cas. Il se produit avec `a*b` sur une ligne qui ne contient que des `a` : depuis chaque position, l'automate lit tous les `a` restants en attendant un `b` qui n'arrive jamais. Une amélioration en $O(\ell)$ est présentée à la section 9.

6.2 Recherche d'un mot simple : algorithme KMP

La méthode naïve place le motif à chaque position du texte et compare caractère par caractère ; en cas de différence, elle avance d'une case et recommence depuis le début du motif, donc elle relit plusieurs fois les mêmes caractères. KMP [5] utilise le fait qu'on connaît déjà les caractères qu'on vient de lire (les premiers du motif) pour **ne jamais reculer dans le texte**, seulement dans le motif. Il calcule d'abord un tableau `lps` (*longest prefix suffix*) : `lps[i]` est la longueur du plus long début du motif qui est aussi une fin de `motif[0..i]`, sans être ce morceau entier. Pour `abab`, `lps` $= [0, 0, 1, 2]$: si `abab` a été lu et que le caractère suivant ne correspond pas, les deux derniers caractères lus forment déjà `ab`, et on reprend à la position 2 du motif sans relire le texte.

```
def construire_lps(motif):
    lps = [0] * len(motif)
    longueur, i = 0, 1
    while i < len(motif):
        if motif[i] == motif[longueur]:
            longueur += 1
            lps[i] = longueur
            i += 1
        elif longueur != 0:
            longueur = lps[longueur - 1]
        else:
            lps[i] = 0
            i += 1
    return lps

def kmp(motif, texte):
    lps = construire_lps(motif)
    i = j = 0          # i : position dans le texte, j : dans le motif
    while i < len(texte):
        if texte[i] == motif[j]:
            i += 1
            j += 1
            if j == len(motif):
                return True
        elif j != 0:
            j = lps[j - 1]     # on recule dans le motif, pas dans le texte
        else:
            i += 1
    return False
```

Complexité. À chaque tour de la boucle, soit i augmente, soit j diminue sans que i bouge, donc $i - j$ augmente. Comme i et $i - j$ ne dépassent pas ℓ, la boucle fait au plus 2ℓ tours : $O(\ell)$. Le même raisonnement donne $O(m)$ pour `construire_lps`. Au total $O(m + \ell)$, contre $O(\ell^2)$ dans le pire cas pour le matcher de l'automate. Remarque : `kmp` reconstruit `lps` à chaque ligne, ce qui ajoute $O(m)$ par ligne ; ce coût est faible pour des motifs courts, mais on pourrait calculer le tableau une seule fois pour tout le fichier.

7 Tests et validation

Nous avons testé chaque module seul (tests unitaires), puis le programme complet comparé à `egrep` (test de conformité). Tous les tests sont écrits avec `pytest` (commande `pytest`).

Tests unitaires. Un test unitaire vérifie une seule étape : s'il échoue, on sait quelle étape contient l'erreur. La table 2 résume les 28 tests. Deux idées les guident : *comparer deux méthodes censées donner le même résultat* (l'AFD et l'AFD minimal doivent accepter exactement les mêmes mots que l'ε-AFN) et *utiliser des cas aléatoires* (pour KMP, 2000 couples (motif, texte) sur les seules lettres `a` et `b` sont comparés à l'opérateur `in` de Python ; cet alphabet crée beaucoup de répétitions comme `abab`, là où le tableau `lps` sert, donc là où une erreur apparaîtrait).

Fichier de test	Tests	Ce qui est vérifié
`test_parser.py`	8	arbre de chaque opérateur, réécriture du `+`, erreurs
`test_nfa.py`	13	gabarits de Thompson, mots acceptés ou refusés
`test_dfa.py`	2	nombre d'états, même réponse que l'ε-AFN
`test_minimize.py`	3	`ab|cb` passe de 5 à 3 états, même réponse
`test_kmp.py`	2	tableau `lps`, comparaison avec Python

TABLE 2 – Tests unitaires

Conformité avec `grep -E`. Les tests unitaires ne vérifient pas le programme complet. Un test lance donc notre programme et `grep -E` sur les mêmes entrées et compare les lignes affichées et le code de retour. Il utilise 11 motifs : `Sargon` (KMP), `S(a|g|r)+on` (exemple du sujet), `Babylon|Assyria`, `(K|k)ing`, `Eu.hrates`, `a.c`, `(ab)*c`, `ab*c`, `Th(e|is)␣` (motif avec un espace), `a*` (mot vide : toutes les lignes) et `xyzxyz` (absent : code 1, KMP) ; les autres passent par l'automate. Les deux fichiers sont `petit.txt` (écrit par nous, avec des cas limites : lignes vides, espaces, lettres répétées) et le livre Gutenberg `56667-0.txt`. Cela fait $11 \times 2 = 22$ comparaisons, toutes identiques. Nous avons aussi vérifié à la main qu'un fichier introuvable et une parenthèse non fermée, par exemple `a(b`, donnent le code 2, comme `egrep`. Le motif vide est un choix volontaire de notre part : nous le refusons (code 2), alors que `egrep` l'accepte et affiche toutes les lignes.

Bug détecté : fins de ligne `\r\n`. Ce test a trouvé un bug que les tests unitaires ne pouvaient pas voir. Sur le livre Gutenberg, `diff` montrait que *toutes* les lignes affichées différaient de celles de `grep`, alors qu'elles semblaient identiques à l'écran. Sous Linux, une ligne se termine par `\n` ; sous Windows, et dans les fichiers Gutenberg, par `\r\n`. Par défaut, `open` de Python remplace `\r\n` par `\n`, donc notre programme affichait les lignes sans le `\r`, alors que `grep` les affiche telles quelles. Nous ouvrons maintenant le fichier avec `newline=""` et retirons seulement le `\n` final : la sortie est identique à celle de `grep`, octet par octet. Le bug n'était dans aucun algorithme, mais dans la lecture du fichier : un test qui compare le programme complet à une référence trouve des erreurs que les tests de chaque module ne voient pas.

8 Évaluation des performances

Nous comparons trois méthodes : notre automate, notre KMP, et la commande `egrep` du système. Le jeu de données est le livre `56667-0.txt` du projet Gutenberg [8], celui de l'énoncé (téléchargé par `tests/download.sh`) ; pour des fichiers plus gros, nous créons des fichiers de 1, 2, 4 et 8 copies du livre. Chaque mesure est répétée 10 fois et nous gardons la *moyenne* ; sur les graphiques, les traits noirs montrent l'*écart-type* (si les traits de deux barres se chevauchent, la différence entre elles n'est pas significative). Pour forcer une méthode, nous appelons directement la fonction de recherche voulue sur le même motif.

Un piège de mesure. Au premier essai, le temps d'`egrep` ne changeait pas quand le fichier grossissait : sa sortie allait vers `/dev/null`, et `grep` détecte ce cas et s'arrête à la première ligne trouvée, puisque personne ne lit le résultat. Nous récupérons maintenant vraiment la sortie. Un benchmark doit être vérifié, pas seulement lancé.

8.1 Expériences 1 et 2 : méthode et taille du fichier

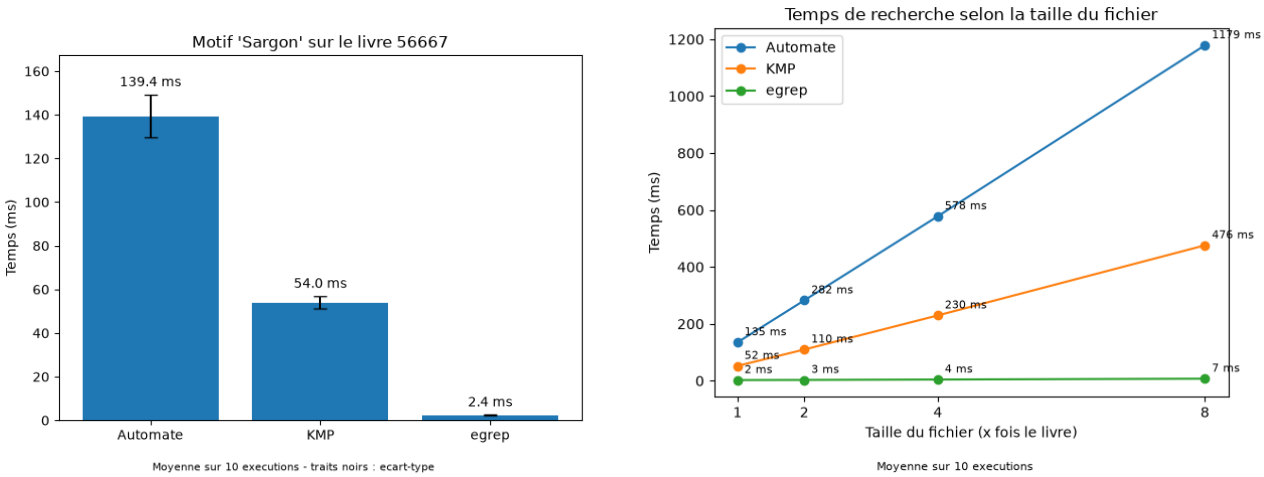


FIGURE 5 – Gauche : recherche de `Sargon` selon la méthode (expérience 1). Droite : temps selon la taille du fichier (expérience 2).

Expérience 1 (motif `Sargon`, figure 5, gauche). KMP est 2,6 fois plus rapide que l'automate (54,0 ms contre 139,4 ms), ce qui confirme le choix de l'utiliser pour les motifs littéraux. `egrep` (2,4 ms) est 58 fois plus rapide que l'automate et 23 fois plus rapide que KMP (section 9).

Expérience 2 (1, 2, 4 et 8 copies, figure 5, droite). Chaque fois que la taille du fichier double, le temps double aussi pour l'automate (135, 282, 578 puis 1179 ms) et pour KMP (52, 110, 230 puis 476 ms) : la croissance est *linéaire*. Le temps d'`egrep` augmente beaucoup moins vite, de 2,5 à 7,4 ms : une grande partie des 2,5 ms est le lancement du programme, un coût fixe ; chaque copie du livre ajoute environ 0,7 ms.

8.2 Expérience 3 : forme de l'expression régulière

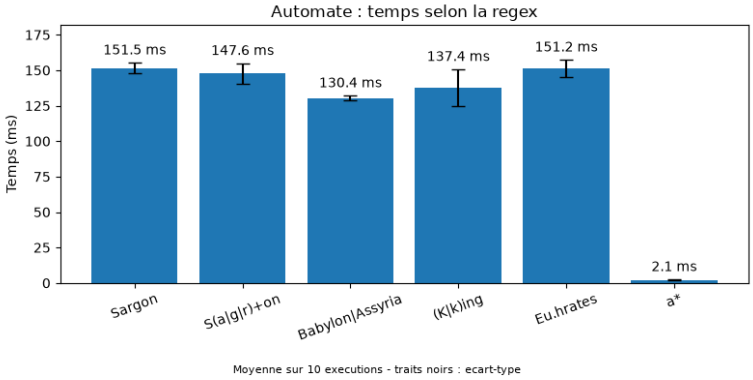


FIGURE 6 – Temps de recherche selon l'expression régulière (automate)

Nous mesurons le temps de l'automate (figure 6) pour six motifs de formes différentes (mot simple, `+`, alternative, parenthèses, point, étoile) ; même `Sargon` passe ici par l'automate, pour comparer toutes les formes avec la même méthode. Toutes les expressions prennent entre 130 et 152 ms, sauf `a*`, qui prend 2,1 ms. Les écarts entre les autres sont faibles : les traits noirs de `Sargon` et de `Eu.hrates` se

chevauchent, et seul `Babylon|Assyria` est un peu plus rapide (130,4 ms, soit environ 15 % de moins). `Sargon` prend ici 151,5 ms contre 139,4 ms dans l'expérience 1 : les deux séries ont été faites à des moments différents, et le temps varie d'environ 10 ms d'une exécution à l'autre.

9 Discussion et limites

Pourquoi KMP est plus rapide que notre automate. Notre matcher relance l'automate à chaque position de la ligne et peut relire plusieurs fois les mêmes caractères. Sur un texte réel, cette relecture reste pourtant rare : avec `Sargon`, l'automate échoue presque toujours dès le premier caractère. La différence vient surtout du travail fait *à chaque position* : le matcher appelle `cherche_depuis` une fois par caractère, et un appel de fonction coûte cher en Python, alors que KMP parcourt la ligne dans une seule boucle.

Pourquoi la croissance est linéaire malgré $O(\ell^2)$. Ce coût concerne *une seule* ligne, où ℓ est sa longueur. Copier le livre ajoute des lignes, mais chaque ligne garde la même longueur : le temps total est proportionnel au nombre de lignes, donc à la taille du fichier.

Pourquoi la forme de l'expression compte peu, et le cas de `a*`. Une fois l'AFD construit, lire un caractère coûte une consultation de table, quelle que soit l'expression ; la forme ne joue que sur la construction, faite une seule fois avant la lecture et très rapide pour des expressions courtes. `a*` accepte le mot vide : l'état initial est déjà final et le matcher accepte chaque ligne sans lire un caractère ; il ne reste que le coût de lecture du fichier.

Pourquoi `egrep` est beaucoup plus rapide. `egrep` est écrit en C, langage compilé, alors que notre code est en Python, interprété et beaucoup plus lent pour une boucle caractère par caractère. Il cherche aussi d'abord les mots fixes du motif avec un algorithme de type Boyer-Moore [6] (qui saute des caractères sans les lire), construit son AFD au fur et à mesure [7] et lit le fichier par gros blocs. Comparer les temps absolus est donc peu utile ; en revanche, la forme des courbes est la même : le temps augmente de façon linéaire avec la taille du fichier, pour nos deux méthodes comme pour `egrep`.

Limites et améliorations possibles.

— **Recherche par automate en une seule lecture.** Au lieu de relancer l'automate à chaque position, chercher `.*R` au lieu de `R` : le `.*` permet de « sauter » n'importe quel début de ligne, l'AFD essaie toutes les positions de départ en même temps et lit la ligne une seule fois. Le coût passe de $O(\ell^2)$ à $O(\ell)$, avec un seul appel de fonction par ligne.

— **Calculer `lps` une seule fois.** Il ne dépend que du motif : on pourrait le calculer avant de lire le fichier.

— **Nœud `PLUS`.** La réécriture $X^+ = XX^*$ copie X ; un nœud `PLUS`, dont le gabarit de Thompson est celui de l'étoile sans la ε-transition de saut, éviterait la copie avec des `+` imbriqués.

— **Détecter plus de motifs littéraux.** `est_litteral` travaille sur la chaîne : `(ab)c` passe par l'automate. Tester l'arbre enverrait vers KMP tout arbre fait seulement de `CHAR` et `CONCAT`.

— **Nombre d'états de l'AFD.** Le pire cas est 2^n états ; un motif comme `(a|b)*a(a|b)(a|b)(a|b)` ferait grossir l'AFD très vite. Une solution, utilisée par `grep`, est de construire les états seulement quand on en a besoin, pendant la lecture du texte [7].

10 Conclusion et perspectives

Nous avons réalisé un clone simplifié de `egrep` en Python, qui accepte les lettres, le point, la concaténation, l'alternative, les parenthèses, l'étoile et le plus. Il suit la méthode du cours : arbre syntaxique, ε-AFN (Thompson), AFD (sous-ensembles), AFD minimal (Moore) ; pour les motifs sans

opérateur, il utilise directement KMP. Les tests unitaires vérifient chaque étape, et le test de conformité compare le programme à `grep -E` sur 11 motifs et deux fichiers : lignes affichées et codes de retour sont identiques dans les 22 cas. Ce test a aussi trouvé un bug de fins de ligne `\r\n`.

Les mesures confirment la théorie : KMP est 2,6 fois plus rapide que notre automate pour un motif littéral ; le temps augmente linéairement avec la taille du fichier ; une fois l'automate construit, la forme de l'expression a peu d'effet. Notre programme reste beaucoup plus lent que `egrep`, écrit en C avec des optimisations avancées. Les améliorations les plus utiles seraient la recherche en une seule lecture avec `.*R` ($O(\ell^2)$ à $O(\ell)$ par ligne), le calcul unique de `lps`, un nœud `PLUS`, la détection des littéraux sur l'arbre et la construction des états de l'AFD à la demande.

Ce projet nous a montré qu'un algorithme correct en théorie ne suffit pas : il faut aussi le tester face à une référence et mesurer ses performances avec soin. Parmi les perspectives, un index des mots du texte (table de hachage ou P-trie) accélérerait les recherches répétées ; nous ne l'avons pas implémenté. Le sujet se limite à l'ASCII et au sous-ensemble ci-dessus : les autres caractères spéciaux d'ERE (`? [ ] { } ^ $` et la barre oblique inverse) sont traités comme des lettres ordinaires. Ces outils serviront pour la recherche dans une bibliothèque de livres (projet 2).

Références

[1] A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman. Compilers : Principles, Techniques, and Tools, 2e éd. Addison-Wesley, 2006.

[2] K. Thompson. Regular expression search algorithm. *Communications of the ACM*, 11(6) :419–422, 1968.

[3] E. F. Moore. Gedanken-experiments on sequential machines. *Automata Studies*, Princeton University Press, p. 129–153, 1956.

[4] J. E. Hopcroft. An $n \log n$ algorithm for minimizing states in a finite automaton. *Theory of Machines and Computations*, Academic Press, p. 189–196, 1971.

[5] D. E. Knuth, J. H. Morris, V. R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2) :323–350, 1977.

[6] R. S. Boyer, J. S. Moore. A fast string searching algorithm. *Communications of the ACM*, 20(10) :762–772, 1977.

[7] R. Cox. Regular Expression Matching Can Be Simple And Fast. 2007. `https://swtch.com/~rsc/regexp/regexp1.html`.

[8] Projet Gutenberg. `https://www.gutenberg.org`.